

```
!pip install boto3
```

```
Collecting boto3
  Downloading boto3-1.40.41-py3-none-any.whl.metadata (6.7 kB)
Collecting botocore<1.41.0,>=1.40.41 (from boto3)
  Downloading botocore-1.40.41-py3-none-any.whl.metadata (5.7 kB)
Collecting jmespath<2.0.0,>=0.7.1 (from boto3)
  Downloading jmespath-1.0.1-py3-none-any.whl.metadata (7.6 kB)
Collecting s3transfer<0.15.0,>=0.14.0 (from boto3)
  Downloading s3transfer-0.14.0-py3-none-any.whl.metadata (1.7 kB)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in /usr/local/lib/python3.12/dist-packages (from botocore<1.41.0,>=1.40.41)
Requirement already satisfied: urllib3!=2.2.0,<3,>=1.25.4 in /usr/local/lib/python3.12/dist-packages (from botocore<1.41.0,>=1.40.41)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil<3.0.0,>=2.1->botocore<1.41.0,>=1.40.41)
Downloading boto3-1.40.41-py3-none-any.whl (139 kB)
  139.3/139.3 kB 2.8 MB/s eta 0:00:00
Downloading botocore-1.40.41-py3-none-any.whl (14.0 MB)
  14.0/14.0 MB 94.3 MB/s eta 0:00:00
Downloading jmespath-1.0.1-py3-none-any.whl (20 kB)
Downloading s3transfer-0.14.0-py3-none-any.whl (85 kB)
  85.7/85.7 kB 5.4 MB/s eta 0:00:00
Installing collected packages: jmespath, botocore, s3transfer, boto3
Successfully installed boto3-1.40.41 botocore-1.40.41 jmespath-1.0.1 s3transfer-0.14.0
```

```
#!/usr/bin/env python3
"""
prompt_comparison_pipeline.py

Compare multiple prompt patterns using AWS Bedrock Claude 3 Sonnet.
Print outputs side by side and provide simple quality scoring.
"""

import boto3
import json

# ----- AWS Bedrock direct credentials -----
AWS_ACCESS_KEY = "AKIAVSX4Z23PH4R4CRWQ"
AWS_SECRET_KEY = "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"
AWS_REGION = "us-east-1"
MODEL_ID = "anthropic.claude-3-sonnet-20240229-v1:0"

# ----- Prompt variations -----
prompt_variations = [
    {
        "name": "concise_zero_shot",
        "prompt": "Write Python function factorial(n:int)->int. Raise ValueError if n<0."
    },
    {
        "name": "detailed_zero_shot",
        "prompt": "You are a Python developer. Implement factorial(n:int)->int. Include docstring, type hints, minimal comments"
    },
    {
        "name": "few_shot",
        "prompt": """You are a professional Python developer. Example:

Spec: add(a:int, b:int) -> int that returns a+b
Code:
def add(a:int,b:int)->int:
    \"\\\"Return sum of a and b\\\"\\\"
    return a + b

Now implement factorial(n:int)->int. Raise ValueError if n<0.
"""
    }
]

# ----- Invoke Bedrock model -----
rt = boto3.client(
    "bedrock-runtime",
    region_name=AWS_REGION,
    aws_access_key_id=AWS_ACCESS_KEY,
    aws_secret_access_key=AWS_SECRET_KEY
)

# ----- Run variations -----
outputs = []

for var in prompt_variations:
    print(f"\n=== Running Variation: {var['name']} ===\n")
    print("Prompt:\n", var["prompt"])

    body = {
        "anthropic version": "bedrock-2023-05-31",

```

```

        "max_tokens": 500,
        "temperature": 0.2,
        "messages": [{"role": "user", "content": var["prompt"]}]}
    }

    response = rt.invoke_model(
        modelId=MODEL_ID,
        body=json.dumps(body),
        contentType="application/json"
    )

    obj = json.loads(response["body"].read())
    text = obj["content"][0]["text"].strip()

    # Remove code fences if present
    if text.startswith("```"):
        text = text.strip("```").strip()

    # Append output for comparison
    outputs.append({
        "variation": var["name"],
        "prompt": var["prompt"],
        "output": text
    })

# ----- Compare outputs -----
print("\n\n=== Comparison of Prompt Variations ===\n")

for o in outputs:
    print(f"--- Variation: {o['variation']} ---")
    print("Generated Output:\n", o["output"])

    # Simple quality scoring
    score = 0
    if "def factorial" in o["output"]:
        score += 1
    if "raise ValueError" in o["output"]:
        score += 1
    if "\\\"\\\"\\\" in o["output"] or "# " in o["output"]:
        score += 1
    print(f"Simple Quality Score (0-3): {score}/3")
    print("\n-----\n")

=== Running Variation: concise_zero_shot ===

Prompt:
Write Python function factorial(n:int)->int. Raise ValueError if n<0.

=== Running Variation: detailed_zero_shot ===

Prompt:
You are a Python developer. Implement factorial(n:int)->int. Include docstring, type hints, minimal comments. Raise ValueErr

=== Running Variation: few_shot ===

Prompt:
You are a professional Python developer. Example:

Spec: add(a:int, b:int) -> int that returns a+b
Code:
def add(a:int,b:int)->int:
    """Return sum of a and b"""
    return a + b

Now implement factorial(n:int)->int. Raise ValueError if n<0.

=== Comparison of Prompt Variations ===

--- Variation: concise_zero_shot ---
Generated Output:
Here's a Python function that calculates the factorial of a given non-negative integer `n`:

```python
def factorial(n: int) -> int:
 """
 Calculates the factorial of a non-negative integer n.

 Args:
 n (int): A non-negative integer.

 Returns:
 int: The factorial of n.

```

```
Raises:
 ValueError: If n is negative.
"""
if n < 0:
 raise ValueError("n must be a non-negative integer")

if n == 0:
 return 1

result = 1
for i in range(1, n + 1):
 result *= i

return result
...
```